

---

# AutoMarket

## Full Stack Web Application

### Technical Documentation

---

MEAN Stack | MongoDB | Express | Angular | Node.js  
VMware Workstation | 4 Virtual Machines | Ubuntu Desktop  
Newark, NJ | April 2026

# 1. Project Overview

AutoMarket is a full-stack car marketplace web application built on the MEAN stack (MongoDB, Express, Angular, Node.js). The application allows customers to browse, search, and purchase vehicles, while dealers can list cars with image uploads. The platform is deployed across four virtual machines running on VMware Workstation, simulating a production enterprise network architecture.

## 1.1 Features

- User registration and login with JWT authentication
- Customer and dealer account types with role-based access
- Car inventory with search by make, model, year, and VIN
- VIN decoder using the NHTSA VPIC public API
- Dealer dashboard with Cloudinary image upload
- Shopping cart that persists for both guests and logged-in users
- Checkout system that saves orders to MongoDB
- Dealer review system for customers
- Email notifications via Nodemailer and Gmail
- Financing calculator with real-time payment estimates
- Centralized logging with rsyslog

## 1.2 Technology Stack

| Layer          | Technology          | Version    | VM          |
|----------------|---------------------|------------|-------------|
| Frontend       | Angular             | 21.x       | Frontend VM |
| Backend        | Node.js + Express   | 20.x / 4.x | Backend VM  |
| Database       | MongoDB + Mongoose  | 7.0 / 8.x  | Database VM |
| Logging        | rsyslog             | 8.x        | Backend VM  |
| Auth           | JWT + bcryptjs      | 9.x / 2.x  | Backend VM  |
| Images         | Cloudinary + Multer | 2.x / 1.x  | Backend VM  |
| Email          | Nodemailer + Gmail  | 6.x        | Backend VM  |
| VIN API        | NHTSA VPIC          | Public     | External    |
| Virtualization | VMware Workstation  | 17.x       | Host        |

## 2. System Architecture

---

The application follows a three-tier architecture distributed across four virtual machines connected via a private VMware host-only network (VMnet2). Each VM has a distinct role and communicates only with the VMs it needs, implementing network isolation for security.

### 2.1 Network Topology

| VM          | Private IP    | Role        | External Access |
|-------------|---------------|-------------|-----------------|
| Frontend VM | 192.168.50.10 | Angular app | NAT (internet)  |
| Backend VM  | 192.168.50.20 | Express API | Private only    |
| Database VM | 192.168.50.30 | MongoDB     | Private only    |
| Logging VM  | 192.168.50.40 | Reserved    | Private only    |

The Frontend VM is the only VM with internet access (dual NIC: NAT + VMnet2). The Backend and Database VMs communicate exclusively on the private 192.168.50.0/24 subnet, isolating sensitive data from external networks.

### 2.2 Request Flow

Every user interaction follows this path through the system:

1. User opens browser on Frontend VM at localhost:4200
2. Angular makes HTTP request to Backend VM at 192.168.50.20:5000/api/...
3. JWT Auth Interceptor automatically attaches Bearer token to every request
4. Express validates the token, processes the request, queries MongoDB
5. MongoDB on Database VM (192.168.50.30:27017) returns data
6. Express sends JSON response back to Angular
7. Angular renders the UI and updates the view

## 3. Virtual Machine Setup

---

All four VMs run Ubuntu Desktop 22.04 LTS on VMware Workstation. The network was configured using VMware's Virtual Network Editor with VMnet2 set to Host-only mode.

### 3.1 VMnet2 Configuration

- Type: Host-only (no external routing)
- Subnet IP: 192.168.50.0
- Subnet mask: 255.255.255.0
- DHCP: Disabled (static IPs assigned manually)

### 3.2 VM Network Configuration

Each VM uses Netplan for static IP assignment. The Frontend VM has two network adapters (NAT for internet and VMnet2 for the private network). The Backend and Database VMs have only VMnet2 after setup, isolating them from the internet.

```
/etc/netplan/01-netcfg.yaml - Frontend VM
network:
  version: 2
  ethernets:
    ens33:
      dhcp4: true          # NAT - internet access
    ens36:
      dhcp4: false
      addresses:
        - 192.168.50.10/24 # Private network
```

## 4. Database — MongoDB

MongoDB 7.0 runs on the Database VM (192.168.50.30). It is configured with authorization enabled, binding only to the private network IP so only the Backend VM can connect.

### 4.1 Configuration

```
/etc/mongod.conf
net:
  port: 27017
  bindIp: 127.0.0.1,192.168.50.30
security:
  authorization: enabled
```

### 4.2 Database and User

- Database name: automarket
- Application user: amuser (readWrite role on automarket database)
- Admin user: admin (userAdminAnyDatabase)
- Connection string:  
mongodb://amuser:password@192.168.50.30:27017/automarket?authSource=automarket

### 4.3 Collections

| Collection | Purpose                      | Key Fields                                            | Relations                                 |
|------------|------------------------------|-------------------------------------------------------|-------------------------------------------|
| users      | Customer and dealer accounts | username, email, password, account_type               | Referenced by cars, reviews, cart, orders |
| cars       | Vehicle listings             | make, model, year, price, mileage, vin, image, status | dealer ref to users                       |
| reviews    | Dealer reviews by customers  | rating, comment, createdAt                            | dealer, customer ref to users             |
| cart       | Shopping cart items          | carId, qty, sessionId                                 | user ref to users, car ref to cars        |
| orders     | Completed purchases          | totalPrice, status, createdAt                         | user, car refs                            |

### 4.4 Security

- Passwords are never stored in plain text
- bcryptjs hashes passwords with cost factor 12 before saving (pre-save Mongoose hook)
- Authorization is enabled — only authenticated users can query the database
- MongoDB only accepts connections from 127.0.0.1 and 192.168.50.30 (Backend VM)

- No SQL injection risk — Mongoose validates and casts all input before querying

## 5. Backend — Node.js + Express

The Express API server runs on the Backend VM (192.168.50.20:5000). It handles all business logic, authentication, data validation, and communication between the frontend and database.

### 5.1 Project Structure

```
automarket-api/  
  server.js          - Express app entry point  
  .env              - Environment variables  
  models/  
    User.js         - Mongoose user schema with bcrypt  
    Car.js          - Vehicle listing schema  
    Review.js       - Dealer review schema  
    Cart.js         - Shopping cart schema  
    Order.js        - Purchase order schema  
  routes/  
    auth.js         - Register, login, get me  
    cars.js         - CRUD + search + VIN decode  
    reviews.js     - Get and post reviews  
    cart.js         - Cart management  
    orders.js      - Place and retrieve orders  
  middleware/  
    auth.js         - JWT verification middleware  
  config/  
    cloudinary.js   - Cloudinary + Multer setup  
    mailer.js       - Nodemailer Gmail configuration  
  logs/  
    app.log         - HTTP request logs (Morgan)
```

### 5.2 Security Implementation

- — Sets secure HTTP headers (XSS protection, HSTS, CSP)
- — Validates all POST request bodies before processing
- — Hashes passwords server-side with cost factor 12
- — Stateless authentication with 7-day expiry tokens
- — Configured to allow only the Frontend VM origin
- — Prevents NoSQL injection through type casting and validation

### 5.3 Process Management

PM2 manages the Node.js process on the Backend VM, providing automatic restarts on crash, startup persistence across reboots, and centralized log management.

```
pm2 start server.js --name automarket-api  
pm2 startup      # Auto-start on boot  
pm2 save         # Save process list
```





## 6. API Endpoint Reference

Base URL: <http://192.168.50.20:5000/api> — All protected endpoints require the Authorization: Bearer <token> header, automatically handled by the Angular JWT interceptor.

### 6.1 Authentication Endpoints

| Method | Endpoint       | Auth Required | Description                                                                         |
|--------|----------------|---------------|-------------------------------------------------------------------------------------|
| POST   | /auth/register | No            | Create account. Body: {username, email, password, account_type}. Returns JWT token. |
| POST   | /auth/login    | No            | Login. Body: {email, password}. Returns JWT token and user object.                  |
| GET    | /auth/me       | Yes           | Returns current authenticated user details.                                         |

### 6.2 Cars Endpoints

| Method | Endpoint        | Auth Required | Description                                                               |
|--------|-----------------|---------------|---------------------------------------------------------------------------|
| GET    | /cars           | No            | Returns all available cars with dealer info populated.                    |
| GET    | /cars/:id       | No            | Returns single car by MongoDB ObjectId.                                   |
| GET    | /cars/search?q= | No            | Searches cars by make, model, VIN, or trim using regex.                   |
| GET    | /cars/vin/:vin  | No            | Decodes VIN using NHTSA VPIC API. Returns make, model, year, trim.        |
| POST   | /cars           | Dealer        | Creates car listing. FormData with optional image uploaded to Cloudinary. |
| DELETE | /cars/:id       | Dealer        | Deletes car listing. Only the dealer                                      |

| Method | Endpoint | Auth Required | Description                |
|--------|----------|---------------|----------------------------|
|        |          |               | who created it can delete. |

## 6.3 Reviews, Cart, and Orders

| Method | Endpoint           | Auth Required | Description                                                                 |
|--------|--------------------|---------------|-----------------------------------------------------------------------------|
| GET    | /reviews/:dealerId | No            | Returns all reviews for a dealer sorted by newest first.                    |
| POST   | /reviews           | Customer      | Posts a review. Body: {dealerId, rating, comment}. Dealer accounts blocked. |
| GET    | /cart              | Yes           | Returns user's cart items with car details populated.                       |
| POST   | /cart              | Yes           | Adds item to cart. Body: {carId, qty}.                                      |
| DELETE | /cart/:carId       | Yes           | Removes specific car from user's cart.                                      |
| GET    | /orders            | Yes           | Returns user's order history sorted by newest first.                        |
| POST   | /orders            | Yes           | Places order. Body: {carId, totalPrice}. Marks car as sold. Triggers email. |

## 7. External API Integrations

---

### 7.1 NHTSA VIN Decoder

The National Highway Traffic Safety Administration provides a free public API for decoding Vehicle Identification Numbers (VINs). AutoMarket uses this API in two places: the Inventory page VIN decoder tool and the Dealer Dashboard auto-fill feature.

- Endpoint:  
`https://vpic.nhtsa.dot.gov/api/vehicles/DecodeVinValuesExtended/{VIN}?format=json`
- No API key required — completely free and public
- Returns make, model, year, trim, body class, engine model
- Used server-side at GET `/api/cars/vin/:vin` (Backend VM proxies the request)
- Also called directly from Angular as a fallback if backend is unavailable

### 7.2 Cloudinary Image Upload

Cloudinary is a cloud-based media management service. Dealers upload car images through the Dealer Dashboard, which sends the image as FormData to the Express API. The backend uses multer-storage-cloudinary to automatically upload the image to Cloudinary and store the resulting CDN URL in MongoDB.

- Images stored in the 'automarket' folder on Cloudinary
- Images automatically resized to max 1200x800px on upload
- Accepted formats: jpg, jpeg, png, webp
- CDN URL stored in the car document's 'image' field in MongoDB
- Configuration stored in .env: CLOUDINARY\_CLOUD\_NAME, CLOUDINARY\_API\_KEY, CLOUDINARY\_API\_SECRET

### 7.3 Nodemailer + Gmail

Nodemailer handles transactional email notifications using Gmail as the SMTP provider. Gmail App Passwords are used instead of the account password for security. Two email types are sent automatically:

- — **Sent immediately after successful user registration**
- — **Sent after checkout, includes car details and total price**
- Configuration: EMAIL\_USER (Gmail address) and EMAIL\_PASS (16-character App Password) in .env
- Email sending errors are caught and logged but do not block the main operation

## 8. Frontend — Angular 21

The Angular 21 application runs on the Frontend VM at localhost:4200. It uses the standalone component architecture (no NgModule), lazy-loaded routes, and Angular signals for reactive state management.

### 8.1 Project Structure

```
src/app/
  app.ts           - Root component with router outlet
  app.routes.ts    - Lazy-loaded route definitions
  app.config.ts    - App providers (zoneless, HTTP, interceptor)
  pages/
    home/          - Landing page with hero, cards, FAQ, contact
    inventory/      - Car grid with search, sort, VIN decoder
    car-detail/     - Single car view with reviews
    cart/           - Shopping cart with quantity management
    checkout/       - Order form with backend submission
    dealer-dashboard/ - Dealer listing management
    login/          - JWT login form
    register/       - Account creation form
    financing/      - Loan payment calculator
  shared/
    navbar/         - Responsive nav with auth state
    footer/         - Site footer with links
  services/
    auth.service.ts - Login, register, JWT, user signal
    cars.service.ts - Car CRUD, search, VIN decode
    cart.service.ts - Cart signal with localStorage sync
    reviews.service.ts - Get and post reviews
    orders.service.ts - Place and retrieve orders
  interceptors/
    auth.interceptor.ts - Auto-attach JWT to all requests
```

### 8.2 State Management

Angular 21 uses signals for reactive state. The AuthService exposes a currentUser signal that the Navbar reads directly — no subscriptions needed. The CartService uses a computed signal for the cart count badge. Change detection is triggered manually with ChangeDetectorRef.detectChanges() after async operations since zoneless change detection is used.

### 8.3 Authentication Flow

8. User submits login or register form
9. AuthService calls POST /api/auth/login or /register
10. Backend validates credentials, returns {token, user}

11. Token stored in localStorage as 'am\_token', user stored as 'am\_user'
12. currentUser signal updated — Navbar instantly reflects login state
13. All subsequent HTTP requests automatically include Bearer token via interceptor

## 9. Application Pages

---

### 9.1 Home Page (/home)

The landing page features a hero section with a full-bleed background image, a three-card grid showcasing AutoMarket's value propositions, a featured vehicle section, customer testimonials, an FAQ accordion, and a contact section. All navigation uses Angular Router for client-side routing.

### 9.2 Inventory Page (/inventory)

Displays all available cars fetched from MongoDB via GET /api/cars. Features include real-time search that filters the car grid, sorting by price, year, and mileage, and the NHTSA VIN Decoder tool. Each card shows the car image, year, make, model, price, and mileage with View and Add to Cart buttons. Clicking View navigates to the Car Detail page.

### 9.3 Car Detail Page (/car/:id)

Shows full details of a single car including image, all specifications in a key-value grid, and dealer information. Customers can add the car to their cart and leave reviews for the dealer. Reviews are fetched from MongoDB via GET /api/reviews/:dealerId and the average rating is calculated and displayed. Only customer accounts can post reviews — dealer accounts are blocked.

### 9.4 Cart Page (/cart)

The cart persists in localStorage for both guests and logged-in users. Items show the car image, name, VIN, dealer, and price. Users can update quantity or remove items. The order summary calculates subtotal, NJ state tax (6.625%), and total in real time. The Checkout button is disabled when the cart is empty.

### 9.5 Checkout Page (/checkout)

Collects customer information and payment details (demo). On submission, the app calls POST /api/orders for each cart item. The backend marks each car as sold, clears the cart item from MongoDB, and triggers a Nodemailer confirmation email. The cart is cleared in localStorage and the user is redirected to the home page.

### 9.6 Dealer Dashboard (/dealer)

Accessible only to dealer accounts — non-dealers see an access denied message with links to login or register. Dealers can enter a VIN to auto-fill make, model, year, and trim via the NHTSA API. They upload a car image (sent to Cloudinary) and fill in price, mileage, and description. The

listing is saved to MongoDB and appears immediately in the Inventory. Dealers can also remove their own listings.

## 9.7 Financing Page (/financing)

A fully client-side loan payment calculator. Users input vehicle price, down payment, loan term (24-72 months), and interest rate. The monthly payment is calculated using the standard amortization formula in real time as values change. The page also shows total interest paid, total cost, and financing tips and requirements.

## 9.8 Login and Register Pages

Both forms include real-time validation, loading states on buttons, and clear error/success messages from the API. The register form lets users choose between customer and dealer account types. After successful registration, a welcome email is sent via Nodemailer. Both forms redirect to the home page on success.

## 10. Logging

---

HTTP request logging is implemented using Morgan middleware on the Backend VM. All requests are logged in the Apache Combined Log Format to both the console (dev format) and a log file.

- Log file location: ~/automarket-api/logs/app.log
- Format: IP, timestamp, method, path, status code, response time, user agent
- rsyslog monitors app.log and captures entries tagged with 'automarket'
- PM2 maintains separate stdout and stderr logs at ~/.pm2/logs/

Example log entry:

```
192.168.50.10 - - [26/Apr/2026:20:36:54 +0000] "POST /api/auth/register HTTP/1.1" 201
842 "-" "Mozilla/5.0"
```



## 11. Testing Results

All features were tested end-to-end with the full VM stack running. The following table summarizes the test results:

| Feature           | Test                                 | Result |
|-------------------|--------------------------------------|--------|
| User Registration | Create customer and dealer accounts  | PASS   |
| Welcome Email     | Nodemailer sends email on register   | PASS   |
| User Login        | JWT token returned and stored        | PASS   |
| Password Hashing  | bcrypt hash stored in MongoDB        | PASS   |
| Inventory Loading | Cars fetched from MongoDB            | PASS   |
| Car Search        | Filter by make, model, VIN           | PASS   |
| Sort Controls     | Price, year, mileage sorting         | PASS   |
| VIN Decoder       | NHTSA API returns vehicle data       | PASS   |
| Car Detail Page   | Single car view with specs           | PASS   |
| Add to Cart       | Cart persists in localStorage        | PASS   |
| Dealer Reviews    | Customer posts review, stored in DB  | PASS   |
| Cart Management   | Update qty, remove items             | PASS   |
| Checkout          | Order saved to MongoDB               | PASS   |
| Order Email       | Confirmation email sent              | PASS   |
| Car Sold Status   | Car marked sold after purchase       | PASS   |
| Dealer Dashboard  | Access restricted to dealers         | PASS   |
| VIN Auto-fill     | NHTSA fills form on dealer dash      | PASS   |
| Image Upload      | Cloudinary stores car image          | PASS   |
| New Listing       | Car appears in inventory             | PASS   |
| Delete Listing    | Car removed from inventory           | PASS   |
| Financing Calc    | Monthly payment calculated live      | PASS   |
| VM Isolation      | Backend/DB unreachable from internet | PASS   |

## 12. Deployment Notes

---

### 12.1 Development vs Production

The current setup runs Angular in development mode (ng serve) and Node.js directly (node server.js or PM2). For production deployment:

- Run ng build --configuration production to compile Angular to static files
- Configure Apache on the Frontend VM to serve the dist/ folder
- Set up Apache reverse proxy to forward /api requests to 192.168.50.20:5000
- Remove NAT adapters from Backend and Database VMs after all packages are installed
- Configure UFW firewall rules on each VM

### 12.2 Environment Variables

The Backend VM uses a .env file at ~/automarket-api/.env. This file must never be committed to version control. Key variables:

- MONGO\_URI — Full MongoDB connection string with credentials
- JWT\_SECRET — Random secret key for signing JWT tokens
- CLOUDINARY\_CLOUD\_NAME, CLOUDINARY\_API\_KEY, CLOUDINARY\_API\_SECRET — Cloudinary account credentials
- EMAIL\_USER — Gmail address for Nodemailer
- EMAIL\_PASS — 16-character Gmail App Password

### 12.3 Database Seeding

A seed script (seed.js) populates the database with sample car listings for demonstration. It creates 6 cars (BMW M3, BMW 325i, BMW 02 Series, Toyota Supra, Nissan Skyline, Honda NSX) assigned to the first dealer account found in the database.

```
cd ~/automarket-api
node seed.js
```